Bit-Exact on Every Public Scenario and About 90× Faster: Rewriting an Agent-Based Market Simulator Under Exact-Trace Constraints

Manan Gupta*
The Harker School, San Jose, CA, USA
mnn@yogins.com

Abstract

Track 3 of the Agenthon 2026 competition ranks faster implementations of an ABIDES limit-order-book simulator, but a scenario counts only if its output matches a frozen reference; for most scenarios that means the same fills, in the same order, at the same prices and sizes. We first optimized the Python code under a measurement-first workflow. That gave 1.495× on one unit under the organizers' timer and suggested that porting only the event kernel would gain little (an estimated 1.23×). We then reimplemented the organizers' Track 3 configuration of ABIDES (upstream kernel, exchange, order book and oracle, plus the adapter's four agent types) in about 5,000 lines of Rust with no external crates. Matching the reference byte for byte required reproducing behavior the reference never documents: simultaneous events are ordered by sender and recipient rather than by arrival, every send under a stochastic latency model consumes one draw from a shared latency stream, and seed draws whose results are never used still shift later random draws. The engine reproduces all 95 public scenario files byte for byte, passes the official regression suite, and matched the reference on 849 fuzzed scenarios; a subsequent, wider fuzz found one divergence outside the public set, which a later version fixes. In a paired, interleaved benchmark it is 94× faster end to end (geometric mean over the 95 files; 95% CI 89 to 98; 65× to 200× per file). Injecting 18 naive alternatives shows which of these behaviors the official checks detect: an arrival-order queue changes every trace and fails every exact-tier check, yet passes every statistical-tier check. Finally, aligning our timing span with the reference adapter's raised our self-reported Development score 3.09×, which shows how much a self-reported rate depends on the span.

1 Introduction

Agent-based limit-order-book (LOB) simulators are used to study market microstructure and to train and evaluate trading agents [3–5]. ABIDES is a widely used example: a single-threaded Python discrete-event loop that delivers timestamped messages between an exchange agent and populations of noise, value, momentum and market-making agents. Its public reference implementation [15] is archived and written in pure Python, so throughput limits how much simulated data a study can afford.

Track 3 of Agenthon 2026 turns this into a benchmark [1]. A submission is a container image that runs scenarios offline and writes an event trace and a message ledger. It is ranked by events per second, but only if every scenario passes a semantic regression gate. The exact tier of that gate requires the same row count as the reference, the same fills in the same order at the same prices and sizes, exact event coverage, and Kendall's $\tau \geq 0.999$ [17] on event order, with a ± 1 μs timestamp tolerance.

Message latencies are random, so any change to the order in which random numbers are drawn shifts arrival order at the exchange, and a different arrival order can match different orders. Ordinary speedups are therefore unavailable: a faster generator, a reordered floating-point sum, a queue that breaks ties differently, or skipping work whose result is discarded. A faithful reimplementation has to reproduce every incidental choice of the Python code that feeds a random draw or an ordering decision.

*The author directed the project, led the implementation, and guided high-level decisions. Anthropic's Claude was used to assist with code implementation and grammar-level proofreading of the manuscript. The author takes full responsibility for the paper and its results.

We report two stages. The first optimized the Python stack and built the measurement tools; its profile suggested that a partial port could not pay off. The second replaced the stack with a compiled engine, qfsim, checked against the reference layer by layer. The contributions are:

(1) A workflow for optimization under an exact-output constraint, which applies differential testing to a stochastic simulator: an equivalence harness, paired timing, profile attribution, and a ladder of comparisons from single random draws up to whole files (Section 3).

(2) A catalog of implicit behaviors that a faithful reimplementation must reproduce, measured by injecting a naive alternative for each and recording which output files change and which official checks notice (Section 4).

(3) The engine and its evaluation: byte identity on the 95 public files, the official suite and a differential fuzz, and a paired speed benchmark with confidence intervals, a fixed-plus-marginal cost model and a single-core measurement (Sections 5 and 6).

(4) Observations on benchmark validity: a silent 1000× shrinkage of our own development workload that only a realism check caught, and the sensitivity of self-reported throughput to the choice of timing span (Section 7).

The author directed the project, led the implementation, and guided high-level decisions, with assistance from Anthropic's Claude for code implementation and grammar-level proofreading, as described in the acknowledgments. The author takes full responsibility for the paper and its results.

2 Background and related work

The reference. The organizers' baseline image applies four small overlay patches (an order-size model without a heavy dependency, a kernel message ledger, exchange protocol enforcement with self-trade prevention, and a scheduled oracle jump) and a scenario adapter to upstream ABIDES at commit `f9cbe513`, with Python 3.11, numpy 1.26.4, pandas 1.5.3 and pyarrow 15.0.2 pinned. The adapter reads a JSON scenario, seeds a global numpy generator, draws child seeds for the oracle, the exchange, each agent, the latency model and the kernel, runs the kernel, and writes `trace.parquet` (order and quote events) and `message_trace.parquet` (one row per delivered message).

Terms, gates and metric. A *unit* is what the scorer times. The public roster has 71 units: 65 single-market units and 6 batch units that bundle 30 independent sub-markets. That makes 95 *scenario files*, and we report per-file results. The regression suite has 65 scenarios, 42 in the exact tier (Tier A) and 23 in the statistical tier (Tier B). Tier B compares return distributions (Kolmogorov–Smirnov distance at most 0.08) and the time-averaged spread (within ±10 basis points). A stylized-fact gate compares return shape, volatility clustering, the Hill tail index [13] and book depth with the reference [6, 26]; the documentation applies it to the calibration scenarios, and the public regression harness applies it to every scenario. Every regression scenario must pass. The checks compare values, not file bytes. A unit's rate is $n_{\text{events}}/t_{\text{wall}}$, and the score is the arithmetic mean over the roster of unit rates, each clipped at a ceiling derived from 10^7 events/s per market. During Development the rate is the submission's own report, checked for consistency and plausibility. In the planned Final, the organizers' runner times repeats itself, requires every repeat to reproduce the same output bytes, and takes the median as the unit's rate. Two timers matter here. The reference adapter reports the span of `abides.run(config)` for a single unit and the loop over sub-markets for a batch; the organizers' local timer measures the whole `docker run`.

Related work. Checking a new implementation against an existing one on the same inputs is differential testing [22]; Csmith applied it at scale to compilers with random programs [27], and translation validation checks each translation against its source [23]. Our verification ladder applies these ideas to a stochastic simulator whose random streams must be reproduced exactly. Bit-exact replay depends on numpy's policy of freezing the legacy `RandomState` streams [18] and on the floating-point subtleties surveyed by Goldberg [10]. Faster LOB simulators exist when exactness is not required; JAX-LOB, for example, runs order books on GPUs for reinforcement learning [8]. Our constraint is specific to a benchmark that fixes the reference behavior.

3 Method

3.1 Measurement-first optimization of the Python stack

We first optimized the Python code in place, because every change there could be checked against the unmodified program. Four tools shaped the work.

Dispatch-trace equivalence. The harness hashes every call into agent code (time, recipient and message contents), with process-global counters renumbered by first appearance. Identical seeds in separate processes gave identical digests, and different seeds diverged at event 1117.

Paired timing. Before the timebase fix below, the relative standard deviation within a block of runs on our 4-vCPU machine was 7.3%, and one build measured 15,950 and later 20,478 messages/s in the same session with no code change. Our harness therefore alternates A and B in fresh subprocesses and reports the median per-pair ratio with the number of pairs favoring B, a sign test that needs no distributional assumption [see 9, 16]. Every landed patch was measured with at least eight pairs.

Profile attribution. We bucket cProfile self-time by which rewrite would remove it (event loop, latency model, order book, agent logic, copying, standard library, pandas and numpy) and report $1/(1-s)$ for a bucket share s [2]. Self-time excludes callees, so these figures rank candidates; they are estimates, not bounds.

Landing rule. A change landed only with both a paired speedup and a passing equivalence check. The rule rejected one change that was output-identical on the reference unit: a shallow-copy path for logged order dictionaries measured 1.027× over six pairs, inside the noise.

The Python work produced five patches: a bare `heapq` in place of a lock-guarded `queue.PriorityQueue`, a cached agent lookup, removal of abstract base classes that only slowed `isinstance`, incrementally maintained price-level totals, a cache for a pure timestamp formatter, and three small fixes in teardown, copying and clipping. A message-count smoke check (428,939 against the expected 472,869) caught two leaks in the incremental level totals before any trace was compared. On the organizers' timer, on unit `as06`, the stack raised the median rate from 6,198 to 9,263 events/s (1.495×), and all 65 public scenarios passed. Running the sub-markets of a batch unit in separate processes, which is legal because each must reproduce its solo trace, gave a median of 2.58× on the six batch units with all 30 sub-traces byte-identical. We give single measured ratios as recorded (at most three significant figures) and roster summaries above 10× as whole multiples, with confidence intervals where we have repeats.

3.2 A validity failure in our own workload: the 1000× timebase

Upstream ABIDES converts time strings with `pd.to_timedelta(s).to_timedelta64().astype(int)`. Under pandas 3, which infers a microsecond resolution when it parses such strings, that expression returns microseconds rather than nanoseconds; pandas 1.5.3 and 2.2.3 return nanoseconds. Our development environment used pandas 3.0.5, so every derived constant came out 1000× too small against raw nanosecond constants elsewhere. Nothing raised an error. The simulator ran a shorter market: the "full trading day" covered about 23 seconds.

For nearly three weeks every measurement was internally consistent. Traces were deterministic, paired ratios were unanimous and profiles reproduced. None of these checks can detect a wrong workload, because each compares the simulator with itself. The defect surfaced when we built the realism half of the gate and it reported a "full day" that left 456 orders in the book spanning 18 seconds. After the fix the same extractor reported 390 one-minute bars, 18,581 trades, excess kurtosis of 11.0 in bar returns and lag-1 autocorrelation of absolute returns of 0.106: heavy tails and short-range volatility clustering, two signatures of real markets [6]. The organizers pin pandas 1.5.3, so the defect affected only our own workload, but it would have sent the optimization effort to the wrong places (Table 1).

Table 1: Our development workload (upstream `rmsc04`, seed 1, numpy 2.4.6 and pandas 3.0.5) before and after the timebase fix. Estimates are $1/(1-s)$ over cProfile self-time shares s.

Quantity	compressed day	true workload
Messages per session	28,734	472,869
Order-book share of the event-queue phase	2.8%	12.2%
Estimate for a kernel-only compiled port	1.34×	1.23×
Estimate for a kernel plus order-book port	1.39×	1.45×
Estimate for porting all but the standard library and pandas	4.02×	3.8×

3.3 Why we rewrote the whole stack

Attribution estimated that porting the event loop, latency model and core package would gain 1.23× on the true workload, and adding the order book 1.45×, on upstream `rmsc04` rather than the Track 3 roster. These are not bounds: our fourth Python patch, inside the kernel-only bucket, alone measured 1.240×, because self-time undercounts C-level work in callees. Even a generous correction left a partial port far short of an order of magnitude, and a hybrid also pays a Python boundary crossing per event. The full rewrite delivered far more than the 3.8× estimated for porting everything but the standard library and pandas, because self-time buckets also miss interpreter dispatch and allocation. The profile ruled out a partial port correctly but underestimated the full one.

3.4 The verification ladder

Byte identity is stronger than the gate requires, but it is the cheapest complete test oracle: one SHA-256 comparison per file implies every gate property. A hash says nothing about where a run went wrong, however, and one wrong draw corrupts everything after it. We therefore built the rewrite against a ladder of comparisons, in the order the specification prescribes: numpy's random calls against golden files (L0), every global seed draw (L1), every queue push and pop (L2), every draw on every stream as exact bits (L3), the hashes of both output files (L6), the official suite (L7), the seed override that the organizers' local timer uses for its repeats (L8), and a differential fuzz. The specification's oracle (L4) and order-book (L5) rungs were covered by L3 and L6. The reference side is a "lab": pristine ABIDES with the four overlay patches, the adapter and the exact pins, which reproduces all 95 shipped reference files byte for byte on our machine. An observation-only dumper records every draw on every random stream (method, arguments and result as exact doubles) and every queue push and pop as $(t, \text{sender}, \text{recipient}, \text{class}, \text{message id})$; its output files are byte-identical to the references (checked on `as01`). The engine emits the same records behind a flag, so a divergence is located at the first differing draw or pop.

4 What exact equivalence requires

Before writing the engine we wrote a specification: a 1,540-line document that cites the reference source line by line. Its claims came from reading the code, and many were confirmed against instrumented reference runs. The specification was committed complete before the first engine commit and never edited afterward; the "Found" column of Table 2 records how each behavior was established. One behavior the specification got wrong was found later by a divergence, and we discuss it at the end of this section.

To turn the catalog into a measured result, we built a scratch copy of the engine in which one environment variable replaces a single faithful behavior with a naive alternative that a careful port might plausibly choose. For each of 18 alternatives we ran all 95 public files and compared both outputs with the references, and ran the 65 regression scenarios through the official Tier A check, the Tier B statistical check (Tier B scenarios only) and the stylized-fact gate, called exactly as the regression harness calls them, and through the unit scorer's two message-ledger checks. With no injection, every file and every check matched. Table 2 gives the results, grouped into five classes.

Three patterns stand out. First, the nine alternatives that change traces change them early (the median first differing row is within the first 2% of the file for all but one) and fail between 3 and 42 of the Tier A scenarios, yet none of the 18 alternatives failed a single Tier B statistical check, and the stylized-fact gate caught at most 29 of the 65 scenarios. Second, five alternatives change only the message ledger, which the trace checks never read; the ledger checks catch one of them (wakeup ids), so four alternatives change output files and pass every check we ran. Third, four alternatives change nothing on the public files, so their correctness rests on the specification and the fuzz alone.

Ordering. The kernel pushes $(t, (\text{sender}, \text{recipient}, \text{message}))$ onto a heap, and a message compares by its id, so the pop order is $(t, \text{sender}, \text{recipient}, \text{id})$. The code comment, and the docstring of our own first Python patch, describe this as first-in, first-out among tied times, which is false whenever the senders differ. Ties are common because agents wake at shared times and rounded or deterministic latencies collide. Popping ties in push order, or by id alone as the comment suggests, changed every public file from its first rows and failed all 42 Tier A scenarios, while passing all 23 Tier B statistical checks. Cancelling a market maker's orders in reverse order, as iterating a hash map might, changed 93 files.

Random-stream use and ids. A single latency stream serves every sender, and under the log-normal, uniform and Pareto models each send consumes exactly one draw, including sends that change nothing downstream. Drawing once for the whole closing-price broadcast changed the ledger of 94 files but no trace, because the broadcast follows the last order. Giving wakeups their own id counter changed every ledger and no trace, while giving a requeued event a fresh id

Table 2: Injecting one naive alternative at a time. "Trace" and "ledger": of the 95 public files, how many change `trace.parquet` or `message_trace.parquet` bytes. "First": median position of the first differing trace row, as a fraction of the file. Gate columns: of the 42 Tier A scenarios, how many fail the exact check; of the 23 Tier B scenarios, how many fail the statistical check; of all 65, how many fail the stylized-fact gate (SF) or the scorer's two message-ledger checks (L). "Found": R, by reading the code; D, confirmed on an instrumented reference run; M, measured on numpy or the host; F, a divergence in a later fuzz. The last row is not an injection but the evaluated build itself.

			95 files			fails the check			
Class	Naive alternative	Found	trace	ledger	first	A (42)	B (23)	SF (65)	L (65)
Ordering	ties popped in push order (FIFO)	R+D	95	95	0%	42	0	29	0
	ties by message id only, as the code comment says	R+D	95	95	0%	42	0	26	0
	market maker cancels in reverse (hash-map) order	R	93	93	1%	36	0	19	0
	events past the stop time dropped after the pop	R	0	0		0	0	0	0
Stream use and ids	one latency draw for the whole closing broadcast	R	0	94		0	0	0	0
	no latency draw for replies after the close	R	0	1		0	0	0	0
	wakeups take ids from their own counter	R+D	0	95		0	0	0	65
	a requeued busy event gets a fresh id	R+D	86	93	1%	30	0	1	43
	polar twin discarded (no Gaussian cache)	R+M	95	95	0%	42	0	29	0
	Gaussian cache cleared by non-normal draws	R+M	95	95	2%	42	0	26	0
Unused draws	unused exchange seed draw removed	R+D	95	95	0%	42	0	28	0
	unused kernel seed draw removed	R+D	7	7	33%	3	0	4	0
	first jump not drawn when the jump rate is 0	R+D	82	82	0%	36	0	24	0
Delay timing	message compute delay charged after the handler	R+D	0	34		0	0	0	0
	exchange starts with zero compute delay	R+D	0	34		0	0	0	0
Floating point	glibc `log` instead of numpy's for μ	M	0	0		0	0	0	0
	round half away from zero	R	0	0		0	0	0	0
	exact int/float comparison in the oracle	R+M	0	0		0	0	0	0
Oracle close	clamp at the exchange's close (the evaluated build)	F	0	0		0	0	0	0

changed 86 traces. The polar normal method [20] caches its second variate, and the cache survives draws of other kinds; discarding the twin, or clearing it on other draws, changed every file.

Draws whose results are unused. The configuration step draws seeds for the exchange and the kernel, and neither seed is ever used. Both draws still advance the global stream. The exchange's draw precedes the agents' seeds, so removing it changed all 95 files. The kernel's draw is the last one at configuration time, and after it only the oracle uses the global stream, to draw the time to each jump during the run; removing it changed only the 7 files whose runs cross a jump. The oracle also draws the time to its first jump at start-up even when the jump rate is zero and the jump lies years in the future; skipping that draw changed all 82 files without jumps.

Delay timing and floating point. A busy agent's event is requeued with its original ids; compute delay is charged before message handlers but after wakeup handlers; and the exchange starts at 50 ns and drops to zero inside its first handler. Charging message delay after the handler, or starting the exchange at zero delay, changed the ledgers of 34 files and no trace. Python's `round` rounds half to even on the exact double, while Rust's `f64::round` rounds half away from zero, and `mst < T` in the oracle converts the integer to a double first, unlike Python's exact mixed comparison. Neither alternative, nor dropping events after the stop time, changed a public file.

Host dependence. On AVX-512 hosts, numpy 1.26.4 computes the log-normal location `np.log(mean_ns)` with its own vector kernel, which differs from glibc's `log` by one unit in the last place on 21.5% of the integers from 1 to 200,000. Of the seven latency means in the public roster, only 5,000 is affected, and fifteen units use it. With glibc's value no public file changed (Table 2), and 615 runs of those fifteen units, at their own seeds and 40 derived seeds each, produced identical outputs under both values. The reference output is therefore host-dependent in principle, but the effect was not observable on any run we made. qfsim uses numpy's values for the public means and glibc's otherwise.

Output format. Byte identity of the files also required a Parquet writer that follows parquet-cpp 15.0.2's dictionary, page and statistics rules and a port of the snappy 1.1.10 compressor [11, 25]. The gate would accept any valid file with equal values, so this choice serves the oracle, not admissibility.

A specification error found by divergence. The oracle clamps an observation time to one nanosecond before its own close, which the adapter sets to 16:00, not to the exchange's close. The specification wrote only "`mkt_close`", and the evaluated build used the exchange's close. The two differ only when a value trader acts after the exchange has closed,

which no public file reaches: the evaluated build matches all 95 files and passes the official suite, and the 849-case fuzz did not reach the case either. A later fuzz over the whole admitted envelope found it on a 0.3-second scenario with 30 value traders, and a later version fixes it. It is the most direct evidence we have that byte identity on the public set does not certify an engine beyond it.

5 The engine

At the evaluated commit, qfsim is 4,060 lines of Rust in nine modules plus the 921-line nprng crate, with no dependencies from the Rust package registry. Figure 1 shows its structure. The simulation module follows the Python call graph so that message ids, order ids and random draws are allocated in the same statement order as in the reference.

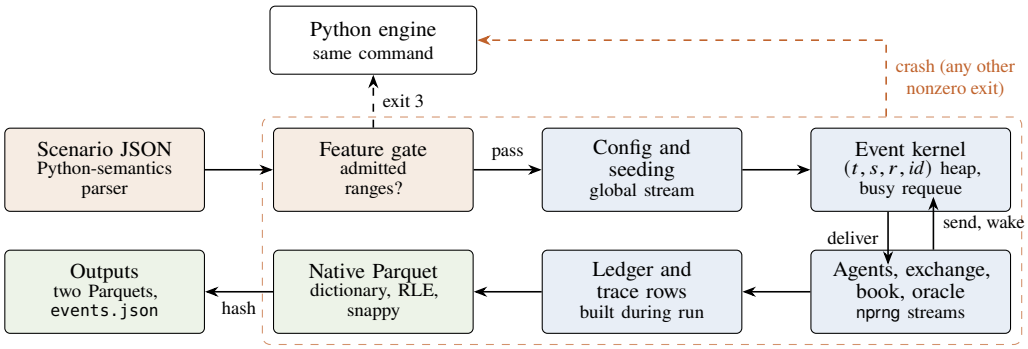


Figure 1: Architecture of qfsim. A scenario outside the admitted ranges exits with code 3 before anything runs; a crash anywhere in the engine (dashed box) exits nonzero. In both cases the image re-runs the identical command in the Python engine. A run that finishes but diverges from the reference exits normally and is not caught.

Random streams. nprng reimplements MT19937 [21] and the frozen legacy distributions of numpy's `RandomState` [12, 18], including mask rejection for bounded integers rather than Lemire's method [19] and the polar normal sampler with its per-stream cache. A golden suite of 276,117 interleaved calls over 13 streams, including a 700-word array seed and blocks that test the Gaussian cache across unrelated draws, matches numpy 1.26.4 bit for bit, final generator state included; a separate golden of 15,000 `pareto` calls also matches. Six injected bugs, among them a disabled Gaussian cache, each fail the suite; three perturbations of constants that only feed accept or reject tests survive it.

Kernel and floating point. The event queue is a binary heap of 128-bit keys that pack the time offset, a 16-bit sender, a 16-bit recipient and a 32-bit message id, so that integer order equals the reference's tuple order; assertions guard every packing bound. The loop reproduces the check-then-pop stop rule, the busy-agent requeue and both charging points for compute delay. All times are 64-bit integer nanoseconds; only the oracle converts them to doubles, where numpy did. The binary links the image's own `libm` dynamically, the library numpy uses in that image, never enables fused multiply-add or fast-math, rounds with `round_ties_even`, and calls `pow` through the C interface so that the compiler cannot fold it.

Output. The reference derives the trace from per-agent event logs with a chain of stable sorts and a quote de-duplication rule. qfsim builds trace and ledger rows during the run and merges them, so no full sort is needed. The Parquet writer encodes column chunks in parallel and hashes them while streaming to disk; its snappy port matches `pyarrow.compress` on 269 of 269 test inputs. A pandas shim, also byte-identical, was replaced because importing pandas and pyarrow alone cost about 0.45 s per process.

Feature gate and fallback. The scenario parser reproduces the adapter's conversions (Python `int`, `float`, truthiness and defaults) and exits with code 3, before writing anything, on unknown keys, non-numeric values where the adapter converts, seeds outside $[0, 2^{32})$, a missing latency configuration, unknown model or policy names, and parameters outside fixed ranges, which in the evaluated build were wider than what its fuzz covered (Section 8). The image entrypoint then re-runs the identical command with the Python engine, as it does after any other nonzero exit. The fallback therefore covers declared-unsupported inputs and crashes. It cannot detect a silent divergence on an admitted scenario, whose risk we bound only empirically (Sections 6 and 8), and a crash after partial work costs the qfsim time plus a full Python run.

6 Evaluation

6.1 Correctness

Table 3 summarizes the evidence. All 95 public files produce both Parquet files byte-identical to the shipped references. The official regression suite passes 65 of 65. On six units chosen to cover jumps, a scheduled shock, Pareto latency and both effective self-trade policies, the golden diff matched every draw, pop and push.

Table 3: Correctness evidence for qfsim. "Identical" means an equal SHA-256 of the file.

Check	Scope	Result
RNG golden (L0)	276,117 calls, 13 streams; 15,000 `pareto` calls	bit-exact
Golden diff (L1 to L3)	6 units; 24,036 to 62,429 draws and 19,094 to 63,777 pops each	identical
Output files (L6)	65 single-market files and 30 batch sub-markets, both files	95/95 identical
Official regression suite (L7)	42 Tier A and 23 Tier B scenarios	65/65 pass
Seed override (L8)	derived seeds on 3 units	identical to lab
Past-stop-time rule	4 constructed scenarios	identical to lab
Differential fuzz	849 compared scenarios (development builds), horizons ≤ 3 s	0 mismatches
Second machine (CI)	all 71 units in the image under the Development limits	95/95 reported trace hashes match

Fuzzing. The fuzzer applies random mutations to public scenarios and compares qfsim with the lab by file hash. Five logged runs cover 867 case indices: 120 cases varied the seed and shortened the horizon, and the rest also drew parameters from small hand-picked sets. The logs record 858 outcomes: 849 compared cases, all byte-identical; 6 that exceeded the lab's time budget; and 3 that crashed the Python reference. The gate rejected none, and 9 indices have no log line. The 401 compared cases of the two earliest runs predate the native Parquet writer and were checked through the pandas shim only; the other 448 checked both writer paths. All five runs used development builds of 26 September that predate the evaluated commit's last performance changes; when the 849 cases were later regenerated, the evaluated build and the later engine produced identical files on every one, with the event counts of the original logs. Every case ran at most 3 simulated seconds, and the mutations stayed close to the public set (Pareto shapes from 0.7 to 3, jump rates up to 10 per second, initial prices up to 200,000, blocks of at most 8 added agents, exchange delays up to 20 μs); the median compared case had about 5,200 events and the largest 2.7 million. The three reference crashes were scenarios that place no orders; qfsim runs them, finds empty output tables and exits with code 3, so the image falls back to Python. This is perturbation testing around the public set, not coverage of everything the gate admits, and it missed the oracle-close error (Sections 4 and 8).

A second machine. The build pipeline pushes the image by digest and then runs every public unit on a GitHub Actions runner under the Development container limits. It fails on any fallback or on any mismatch between the trace hash that qfsim reports and the reference hash in the unit's card. Both recorded runs passed.

6.2 Speed

Method. A paired, interleaved benchmark visited the 95 files in a freshly shuffled order in each of five repeats and, for each file, ran the engines back to back in shuffled order, each as a fresh process. It records end-to-end process time (spawn to exit, with both Parquet files and `events.json` written) and the span each engine reports. The engines were the Python lab (organizers' adapter on the pinned stack), the evaluated qfsim build (commit `3ab4cb2`) with its default threading and pinned to one CPU with a single encoder thread, a later qfsim (commit `8caec5b`, built with its profile-guided optimization data), and, on a stratified subset of 14 files, our optimized Python stack. Every one of the 1,970 runs produced byte-identical outputs. A file's ratio is the median of its five paired ratios; we summarize files by geometric mean and median [7, 14], with 95% two-level bootstrap intervals over files and repeats. The machine was a 4-vCPU KVM guest (Intel Xeon at 2.80 GHz, AVX-512, 15 GB, Ubuntu 24.04, kernel 6.18), with qfsim built by rustc 1.94.1 with the crate's release profile (opt-level 3, LTO, one codegen unit, no CPU-specific flags). Other jobs on the shared machine were idle when the run started and resumed during its last 80 minutes (one-minute load average: median 1.2, range 1.0 to 5.3); the five repeats gave geometric means from 90× to 95×. An earlier run under a load average near 7, with two to four repeats and 17 runs dropped because an unrelated script paused the harness, gave 83× [80, 89].

Table 4: Paired end-to-end speed of qfsim against the Python reference over the 95 public files (five interleaved repeats each; 95% bootstrap intervals in brackets). The cost model fits $t = a + b\,n_{\text{events}}$ to per-file median times with relative-error weights. The last column is a later version measured in the same run.

	evaluated qfsim (`3ab4cb2`)		later (`8caec5b`)
	default threads	one CPU	default threads
Per-file speedup, geometric mean	94× [89, 98]	118× [113, 121]	164× [158, 168]
Per-file speedup, median	91× [83, 98]	119× [111, 123]	
Per-file speedup, minimum and maximum	65× and 200×		
Total time over 95 files (Python: 1485.1 s)	14.0 s	12.2 s	
Fixed cost a (Python: 864 ms)	3.4 ms [1.7, 4.4]	3.9 ms [2.9, 4.7]	4.2 ms [3.7, 4.8]
Marginal cost b per event (Python: 160 μs)	1.92 μs [1.81, 2.06]	1.52 μs [1.47, 1.58]	1.02 μs [0.99, 1.06]
Marginal speedup $b_{\text{Python}}/b_{\text{qfsim}}$ (weighted fit)	84× [77, 90]	105× [101, 110]	157× [150, 165]
Speedup on the reported spans, geometric mean	382× [364, 394]		

Results. Table 4 and Figure 2 give the results. End to end, qfsim is 94× faster than the reference by geometric mean over the files and 106× faster on the total. The per-file ratio depends on size, but not monotonically. It is highest (200×) on the 604-event file, where the reference's start-up (about 864 ms in the fit, mostly interpreter start-up and imports) dominates; it falls to 65× to 120× between about 20,000 and 400,000 events; and it rises to about 125× to 180× on the four files above 500,000 events, where the reference's time per event grows and qfsim's falls. The linear model is a summary, not a law, and its marginal speedup depends on the weighting. With relative-error weights the fit is dominated by the many small files and gives 84× [77, 90] (Table 4); an unweighted fit, dominated by the largest files, gives 159× [101, 178]; and both fits miss the largest files in Figure 2. Pinned to one CPU with one encoder thread, qfsim was faster than with its default threads, so the speedup does not come from parallel hardware; on files this small, starting threads costs more than it saves. The later engine is 1.74× [1.66, 1.81] faster than the evaluated build and 164× faster than the reference, with a marginal speedup of 157× weighted and 228× unweighted. Against our optimized Python stack on the subset, the ratio is 71× [57, 90]; that stack itself is 1.42× faster than the reference end to end.

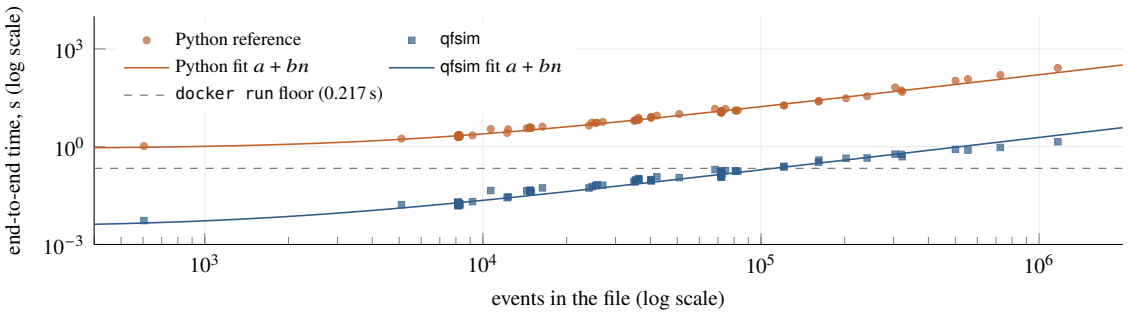


Figure 2: End-to-end time against events for both engines on the 95 public files (medians of five paired repeats), with the fitted fixed-plus-marginal cost models. The dashed line is the measured time of `docker run` for `/bin/true` on the development host, the floor for any container-level measurement.

Inside the container. The organizers' timer measures the whole `docker run`; an earlier measurement (Table 5) shows its 0.217 s floor dominating every unit below about 100,000 events.

6.3 Reported throughput and the timing span

Our first qfsim image reported the span from process start until both Parquet files were written, stricter than the reference adapter's. We then changed the reported span to exactly the adapter's and changed nothing else; the reported trace hashes stayed the same. On the organizers' Development platform the image with the stricter span scored a mean of 2,096,265 events/s over all 71 units, and the image with the adapter's span scored 6,485,109, a factor of 3.09 from the span alone. For comparison, our optimized Python image scored 45,543. In two separate GitHub Actions jobs the two

Table 5: One `docker run` per unit under the Development limits: our optimized Python image against the qfsim image.

Unit	events	Python image (s)	qfsim image (s)	speedup
`s001-price-time-priority`	604	0.941	0.227	4.1×
`as01-base-mix`	24,695	2.454	0.280	8.8×
`sf-01-baseline-regime`	71,932	4.359	0.299	14.6×
`mp05-cancel-churn-newest`	240,829	12.564	0.456	27.6×
`gb-horizon-240s`	555,032	50.222	0.711	70.6×
`gb-mega-throughput`	1,168,360	106.978	0.953	112×

spans gave means of 2.05 and 4.65 million events/s, a factor of 2.27, so the size of the effect depends on the host. All of these are self-reported Development numbers, which the organizers mark as practice feedback rather than ranked results.

7 Lessons for benchmark design

Fix the timing span. The same engine, with identical trace bytes, reported 3.09× different throughput on the organizers' platform depending only on its timing span, and both spans are defensible. Our reported span now matches the organizers' reference adapter. The organizers already plan organizer-measured timing for the Final, and their local timer measures the whole `docker run`; our numbers show why that matters. Their candidate Final policy also requires every timed repeat to reproduce the same output bytes, which a deterministic engine meets at no cost.

Choose the summary statistic deliberately. Once the simulator is fast, container start-up dominates small units under a whole-container clock: in Table 5 the 0.217 s floor turns a 604-event unit into a 4.1× result and a 1.17-million-event unit into a 112× one. An arithmetic mean of per-unit rates weights the fastest units most, while total events over total time weights units by their duration [7, 24]. The choice decides whether the ranking measures simulation or start-up.

Know what each check certifies. In the injection experiment no alternative failed a Tier B statistical check, including both arrival-order queues, which changed every public file from its first rows; the exact tier and, on fewer than half of the scenarios, the stylized-fact gate caught them (Table 2). The public regression harness also applies the exact check to Tier B scenarios, while the unit scorer's semantic check applies only the statistical one. The statistical tier is designed to accept distributionally equivalent engines, so this bounds what a Tier B pass certifies; an injection experiment is a cheap way for a benchmark to measure its own checks.

Properties of the reference stack. Reimplementation surfaced three properties of the reference that a benchmark maintainer may want to know. Its output can depend on the host's vector instruction set through one `np.log` call, although no run we made showed a difference (Section 4). It raises `KeyError` on scenarios that place no orders. And its kernel always writes a summary log to a directory named after the whole-second start time, so concurrent runs started in the same second collide.

8 Limitations

The gate of the evaluated build admits more than its fuzz verified (for example, horizons up to one hour and up to 20,000 agents, against public files that reach 240 s and 144 agents). A silent divergence there, such as the oracle-close error of Section 4, would fail the exact tier, and the fallback does not help. The later engine narrows its gate to the per-parameter ranges covered by a later fuzz of 1,117 scenarios over the admitted envelope (1,019 compared byte-identically), with sessions of up to 6.5 hours and up to 600 agents. For latency means outside the public set the log-normal location comes from glibc and may differ from numpy's by one ulp; 615 runs showed no effect, which bounds the per-run rate at about 0.5% (95% upper limit), not at zero. Our timings come from one shared 4-vCPU machine and one CI runner, the container measurements are single runs, and no organizer-measured Final result was available at the time of writing. The repository is private until the competition ends; the specification and evidence files are available to reviewers on request.

9 Conclusion

The rewrite bought nearly two orders of magnitude once the specification recorded the reference's undocumented behavior, and the injection experiment shows which of it the official checks see.

Acknowledgments

We thank the Agenthon 2026 organizers and track leads for the competition, the public starter materials and regression suite, and their responses to participants' questions.

Use of AI. The author directed the project, led the implementation, and guided high-level decisions. Anthropic's Claude was used to assist with code implementation and grammar-level proofreading of the manuscript. The author takes full responsibility for the paper and its results. The numbers come from logs and evidence files committed to the project repository; the Development scores are the organizers' platform records.

Code availability. The author will release the engine, the Python patches, the harnesses and the recorded measurements under an OSI-approved license after the competition.

References

[1] Agenthon 2026 organizers. Agenthon 2026 Track 3 (Simulation) public starter repository. `https://github.com/Agenthon-2026/track3-simulation-public`, 2026. Toolkit v2.4.4, accessed September 2026.

[2] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*, pages 483–485, 1967.

[3] Selim Amrouni, Aymeric Moulin, Jared Vann, Svitlana Vyetrenko, Tucker Balch, and Manuela Veloso. ABIDES-Gym: Gym environments for multi-agent discrete event simulation and application to financial markets. In *Proceedings of the Second ACM International Conference on AI in Finance (ICAIF)*, 2021. arXiv:2110.14771.

[4] David Byrd, Maria Hybinette, and Tucker Hybinette Balch. ABIDES: Towards high-fidelity market simulation for AI research. *arXiv preprint arXiv:1904.12066*, 2019.

[5] David Byrd, Maria Hybinette, and Tucker Hybinette Balch. ABIDES: Towards high-fidelity multi-agent market simulation. In *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS)*, 2020.

[6] Rama Cont. Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance*, 1(2): 223–236, 2001.

[7] Philip J. Fleming and John J. Wallace. How not to lie with statistics: The correct way to summarize benchmark results. *Communications of the ACM*, 29(3):218–221, 1986.

[8] Sascha Frey, Kang Li, Peer Nagy, Silvia Sapora, Chris Lu, Stefan Zohren, Jakob Foerster, and Anisoara Calinescu. JAX-LOB: A GPU-accelerated limit order book simulator to unlock large scale reinforcement learning for trading. In *Proceedings of the Fourth ACM International Conference on AI in Finance (ICAIF)*, 2023.

[9] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous Java performance evaluation. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, pages 57–76, 2007.

[10] David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.

[11] Google. Snappy: A fast compressor/decompressor. `https://github.com/google/snappy`, 2023. Version 1.1.10.

[12] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585: 357–362, 2020.

[13] Bruce M. Hill. A simple general approach to inference about the tail of a distribution. *The Annals of Statistics*, 3 (5):1163–1174, 1975.

[14] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.

[15] J.P. Morgan Chase & Co. ABIDES (abides-jpmc-public), archived reference implementation. `https://github.com/jpmorganchase/abides-jpmc-public`, 2024. BSD-3-Clause; archived; accessed September 2026; we pin commit f9cbe513.

[16] Tomas Kalibera and Richard Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, pages 63–74, 2013.

[17] Maurice G. Kendall. A new measure of rank correlation. *Biometrika*, 30(1/2):81–93, 1938.

[18] Robert Kern. NEP 19: Random number generator policy. NumPy Enhancement Proposals, `https://numpy.org/neps/nep-0019-rng-policy.html`, 2018.

[19] Daniel Lemire. Fast random integer generation in an interval. *ACM Transactions on Modeling and Computer Simulation*, 29(1), 2019.

[20] George Marsaglia and Thomas A. Bray. A convenient method for generating normal variables. *SIAM Review*, 6(3): 260–264, 1964.

[21] Makoto Matsumoto and Takuji Nishimura. Mersenne twister: A 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1):3–30, 1998.

[22] William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.

[23] Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 1384 of *Lecture Notes in Computer Science*, pages 151–166, 1998.

[24] James E. Smith. Characterizing computer performance with a single number. *Communications of the ACM*, 31 (10):1202–1206, 1988.

[25] The Apache Software Foundation. Apache Parquet file format specification. `https://github.com/apache/parquet-format`, 2026. Accessed September 2026.

[26] Svitlana Vyetrenko, David Byrd, Nick Petosa, Mahmoud Mahfouz, Danial Dervovic, Manuela Veloso, and Tucker Balch. Get real: Realism metrics for robust limit order book market simulations. In *Proceedings of the First ACM International Conference on AI in Finance (ICAIF)*, 2020.

[27] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 283–294, 2011.

A Reproduction commands

Paths are relative to the project repository. The reference lab is built from the pristine upstream tree, the four overlay patches and the exact pins.

```
python3.11 -m venv venv
venv/bin/pip install numpy==1.26.4 pandas==1.5.3 scipy==1.11.4 \
    pyarrow==15.0.2 coloredlogs==15.0.1
cp -r candidate/abides-pristine ab_org
(cd ab_org && for p in order_size_model.pomegranate-free kernel_message_ledger \
     exchange_protocol_stp oracle_scheduled_jump; do
   git apply ../candidate/patches/$p.patch; done)
venv/bin/pip install --no-deps -e ab_org/abides-core -e ab_org/abides-markets
```

The engine, its tests and the paper's experiments:

```
(cd rewrite/nprng && cargo test)             # RNG goldens
(cd rewrite/engine && cargo build --release)
qfsim simulate --config UNIT/scenario.json --out OUT/trace.parquet [--seed N]
qfsim --config UNIT/scenario.json --out-dir DIR --golden run.jsonl
BENCH_SCRATCH=S python paper/evidence/bench_paired.py bench_raw.csv --reps 5
python paper/evidence/analyze_bench.py bench_raw.csv paper/evidence
python paper/evidence/inject_run.py QFSIM_INJECT_BINARY OUT   # after inject.patch
python paper/evidence/analyze_inject.py OUT
```

Tool	Purpose
`rewrite/spec/lab_sweep.py`	lab against the shipped references
`rewrite/spec/golden_dump.py`	golden records from the reference
`rewrite/engine/tools/golden_diff.py`	layers L1 to L3
`rewrite/engine/tools/fuzz.py`	differential fuzz against the lab
`paper/evidence/golden_rerun.sh`	golden diff on six units
`paper/evidence/inject.patch`, `inject_run.py`	semantics-injection experiment
`paper/evidence/bench_paired.py`, `analyze_bench.py`	paired speed benchmark
`paper/evidence/mu_risk.py`, `nplog_vs_glibc.py`	host-dependent logarithm
`.github/workflows/build-image-t3q.yml`	image build and second-machine check